Object-Oriented Analysis of Scikit-learn Library in Python

A Detailed Study of OOP Concepts Using Scikit-learn Source Code

Submitted by:

Name	Student ID
Muhammad Bilal	F25BDATS1M02098
Maryam Tahir	F25BDATS1M02051
Jaweria Zafar	F25BDATS1M02090

Submitted to: Dr.Akmal Shahbaz

Subject: Object-Oriented Programming

Semester: 2nd Semester (M2)

Session: 2026

Department of Data Science
The Islamia University of Bahawalpur

Table of Contents

1. Introduction

One of the most useful paradigms of programming is known as object-oriented programming (OOP). In the contemporary world of software development. It allows the concepts like inheritance, polymorphism, abstraction etc., to be used. Abstraction and Encapsulation, which assist the programmer in producing reusable, modular and maintainable modules. software systems. One of the most widely used machine learning libraries is the Scikit-learn.Scikit-learn is one of the most popular machine learning libraries that is being analysed in this project. Working with machine learning libraries in Python. Scikit-learn has been designed on the basis of robust object-oriented principles. Principles to efficiently and consistently build machine learning models. The main purpose of this project is to study how OOP concepts are implemented in the Scikit learn source code. Different classes/mixins/estimators are studied, to get an understanding of inheritance. The library's architecture, modules and design patterns. To illustrate this, this project contains practical examples and examining the sources to see how the object-oriented design of scikit-learn enables modularity, flexibility and code reuse.

2. About Scikit-learn

The scikit-learn open-source library for machine learning is popularly used with Python. It has simple and Effective tools for data analysis, data mining and machine learning tasks. The structure of the library was created by A reseller of top of NumPy, SciPy, and Matplotlib, as well as other software, which makes it power and easy-to-use for both newbies and experts. professionals.

Scikit-learn is extensively used for the following:

- Classification
- Regression
- Clustering
- Data preprocessing
- Model selection
- Dimensionality reduction

It offers many built-in algorithms including Decision Trees, Random Forests, Support Vector Machines (SVM), and K-Nearest Neighbors (KNN).

Who Uses Scikit-learn?

Scikit-learn is used by:

- Data Scientists
- Machine Learning Engineers
- Researchers
- Students and Educators
- Software Developers

The ability to predictive analysis and recommendations is a task often undertaken by many companies and organizations and that is where scikit-learn comes in. IT's simplicity and reliability make it ideal for systems, fraud detection and data-driven applications.

How to Install Scikit-learn

Scikit-learn can be installed easily using Python’s package manager pip.

Installation Command

```
pip install scikit-learn
```

Verify Installation

After installation, the following command can be used to check the installed version:

```
import sklearn
print(sklearn.__version__)
```

Object-Oriented Design in Scikit-learn

From an OOP point of view, one of the most important features of Scikit-learn is its consistent and well-structured class hierarchy. The library is based on a set of base classes and mixin classes. Local estimators, such as BaseEstimator, ClassifierMixin, RegressorMixin, and TransformerMixin. Every model available in Scikit-learn. These base classes are inherited by all the models, which means they all follow the same format: `fit()`, `predict()`, and `score()`. A perfect example of Inheritance, Polymorphism, and Abstraction in actual real-world python code.

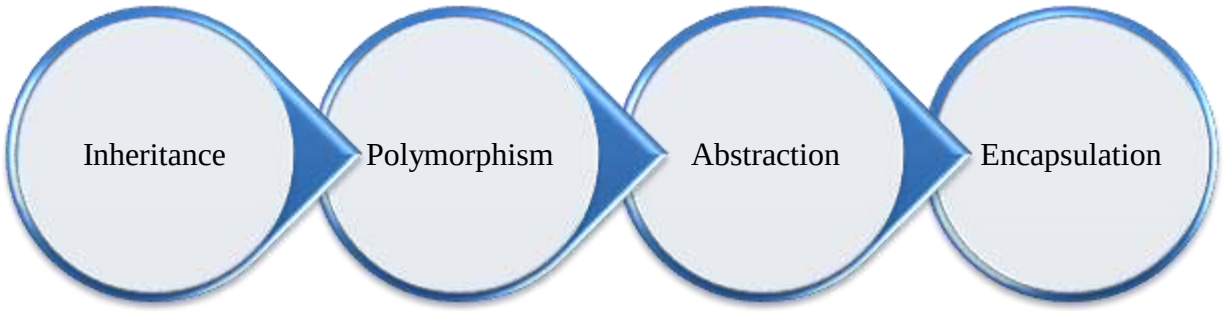
3. Objectives of the Project

Objectives of the project are:

- To read and navigate the professional source code of the Scikit-learn library and understand how it is structured.
- To identify Object-Oriented Programming principles — Encapsulation, Inheritance, Polymorphism, and Abstraction — in the real source code of Scikit-learn.
- To draw and explain a class hierarchy diagram showing the relationships between key classes in Scikit-learn.
- To critically analyze and evaluate the design decisions made by the Scikit-learn library authors and suggest alternatives.
- To compare the OOP approach of Scikit-learn with one alternative library solving the same problem.
- To extend an existing Scikit-learn class by writing our own working Python code that inherits from at least one library class and adds meaningful functionality.

4. OOP Concepts Analysis

Object-Oriented Programming is based on four major pillars: Inheritance, Polymorphism, Abstraction, and Encapsulation. These concepts are widely used in the architecture of the Scikit-learn library and are illustrated in the following diagram.



4.1 Inheritance

Inheritance is an Object-Oriented Programming concept in which a child class acquires the properties and methods of a parent class. This allows code reuse and helps in building a hierarchy of classes. In Python, inheritance is implemented by passing the parent class name inside the parentheses of the child class definition. A child class can also add its own new methods or override the methods of the parent class.

Scikit-learn makes extensive use of inheritance across its source code. The following examples have been identified from the four analysed files.

Example: 01

Single Inheritance File: `base.py` — Line 862

```
class TransformerMixin(_SetOutputMixin):
    """Mixin class for all transformers in scikit-learn."""
```

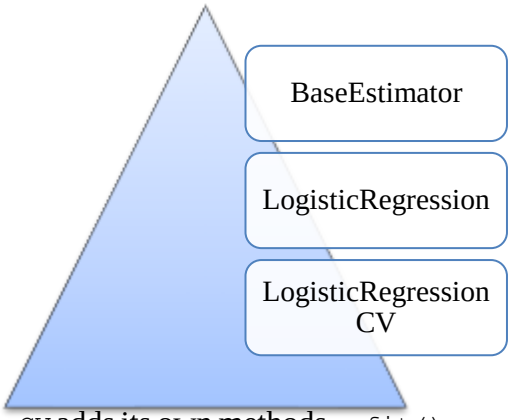
`TransformerMixin` inherits from `_SetOutputMixin`. This is an example of Single Inheritance — one child, one parent. By inheriting from `_SetOutputMixin`, `TransformerMixin` automatically receives the `set_output()` method. It also adds its own method `fit_transform()` which combines `fit()` and `transform()` into one step.

Example 02

Multiple Inheritance File 3: `_logistic.py`— Line 1449

```
class LogisticRegressionCV(LogisticRegression,
                           LinearClassifierMixin,
                           BaseEstimator):
    """Logistic Regression with Cross Validation."""
```

`LogisticRegressionCV` inherits from `LogisticRegression`, which itself is already a child of `BaseEstimator`. This creates a Multi-level Inheritance chain:



`LogisticRegressionCV` adds its own methods — `fit()`, `score()`, and `get_metadata_routing()` — on top of everything it already inherited.

Summary Table

Inheritance Across All 4 Files that is given below:

File	Class	Inherits from	Type
base.py	BaseEstimator	ReprHTMLMixin, _HTMLDocumentationLinkMixin, _MetadataRequester	Multiple
base.py	TransformerMixin	_SetOutputMixin	Single
pipeline.py	Pipeline	_BaseComposition	Single
pipeline.py	FeatureUnion	TransformerMixin, _BaseComposition	Multiple
_logistic.py	LogisticRegression	LinearClassifierMixin, SparseCoefMixin, BaseEstimator	Multiple
_logistic.py	LogisticRegressionCV	LogisticRegression, LinearClassifierMixin, BaseEstimator	Multiple
_base.py	LinearModel	BaseEstimator	Single
_base.py	LinearClassifierMixin	ClassifierMixin	Single
_base.py	LinearRegression	RegressorMixin, MultiOutputLinearModel	Multiple

4.2 Polymorphism

Polymorphism is an Object-Oriented Programming concept that allows the same method name to behave differently depending on the object that calls it. The word "Polymorphism" means "many forms". In scikit-learn, this is primarily achieved through method overriding and a consistent public API (e.g., `fit`, `predict`, `score`). These common methods are implemented within different estimator classes: The names of the methods contain algorithm

specific logic and allow models, transformers and pipelines to be Sony has replaced them in evaluation workflows without any fuss.Sony has seamlessly substituted them with no qualms.

Example: 01

Method Overriding File 1: `base.py`—Line 548 -615

```
class ClassifierMixin:
    def score(self, X, y, sample_weight=None):
        """Return the mean accuracy on the given test data and labels."""
        from sklearn.metrics import accuracy_score
        return accuracy_score(y, self.predict(X),sample_weight=sample_weight)
class RegressorMixin:
    def score(self, X, y, sample_weight=None):
        """Return R coefficient of determination."""
        from sklearn.metrics import r2_score
        y_pred = self.predict(X)
        return r2_score(y, y_pred, sample_weight=sample_weight)
```

The method `score()` behaves differently depending on whether the class is a classifier or a regressor. Both classes have the same method name `score()`, but:

- `ClassifierMixin.score()` calculates accuracy (0 to 1, higher is better)
- `RegressorMixin.score()` calculates R^2 score (1.0 is perfect, can be negative)

This is **polymorphism** — same method name, different behaviour.

Example: 02

Method Overriding File 1: `base.py`

Parent Class (BaseEstimator): Line 524-531

```
class BaseEstimator:
    def __sklearn_tags__(self):
        return Tags(
            estimator_type=None,
            target_tags=TargetTags(required=False),
            transformer_tags=None,
            regressor_tags=None,
            classifier_tags=None,
        )
```

Child Class (ClassifierMixin): Line 582-587

```
class ClassifierMixin:
    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.estimator_type = "classifier"
        tags.classifier_tags = ClassifierTags()
        tags.target_tags.required = True
        return tags
```

Child Class (RegressorMixin):

```
class RegressorMixin:
    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.estimator_type = "regressor"
        tags.regressor_tags = RegressorTags()
        tags.target_tags.required = True
        return tags
```

The sklearn_tags() method is overridden in each child class to return different tags:

- ClassifierMixin → marks estimator as "classifier"
- RegressorMixin → marks estimator as "regressor"
- TransformerMixin → marks estimator as "transformer"

Same method name, different tags returned.

Summary Table:

Example	File	Class	Method

score()	base.py	ClassifierMixin	score()
score()	base.py	RegressorMixin	score()
__sklearn_tags__()	base.py	BaseEstimator	__sklearn_tags__()
__sklearn_tags__()	base.py	ClassifierMixin	__sklearn_tags__()
fit_transform()	base.py	TransformerMixin	fit_transform()
transform()	pipeline.py	Pipeline	transform()
_iter()	pipeline.py	Pipeline	_iter()
fit_transform()	pipeline.py	Pipeline	fit_transform()
predict_proba()	logistic.py	LogisticRegression	predict_proba()
fit()	logistic.py	LogisticRegression	fit()
_predict_proba_lr()	logistic.py	LinearClassifierMixin	_predict_proba_lr()
predict()	_base.py	LinearModel	predict()
decision_function()	_base.py	LinearClassifierMixin	decision_function()

4.3 Abstraction

Abstraction is an Object-Oriented Programming concept that hides complex implementation details and only shows the essential features of an object. In scikit-learn, abstraction is achieved through:

- Design patterns— hiding implementation complexity while providing simple public interfaces
- Abstract Methods — methods declared but not implemented in the base class
- Method Hiding — hiding internal implementation from the user

The user only sees what the method does, not how it does it.

Example: 01

Abstract Base Class Pattern (BaseEstimator) File `base.py (Line 524)`

```
class BaseEstimator(ReprHTMLMixin, _HTMLDocumentationLinkMixin, _MetadataRequester):
    """Base class for all estimators in scikit-learn."""
    def __sklearn_tags__(self):
        return Tags(
            estimator_type=None,
            target_tags=TargetTags(required=False),
            transformer_tags=None,
            regressor_tags=None,
            classifier_tags=None,
        )
```

- BaseEstimator is a concrete class you can instantiate, but it demonstrates abstraction through method hiding — users don't need to understand parameter validation internals
- It provides common methods like `get_params()` and `set_params()` without revealing how they work internally
- Child classes only know they need to implement `fit()` method
- The complexity of parameter validation, serialization, and representation is abstracted away

User view:

```
# User doesn't know or care about internal complexity
model = LogisticRegression()  # Inherits from BaseEstimator
model.get_params()  # Works magically without user knowing implementation
```

Example: 02

Abstract Mixin with required methods File `base.py (Line 582)`

```
class ClassifierMixin:
    """Mixin class for all classifiers in scikit-learn."""
    def __sklearn_tags__(self):
        tags = super().__sklearn_tags__()
        tags.estimator_type = "classifier"
        tags.classifier_tags = ClassifierTags()
        tags.target_tags.required = True
        return tags
```

TransformerMixin with fit_transform abstraction (line 900)

```
class TransformerMixin(_SetOutputMixin):
    def fit_transform(self, X, y=None, **fit_params):
        """
        Fit to data, then transform it.
        This method provides a default implementation that calls fit and then
        transform, but child classes can override it for efficiency.
        """
        if y is None:
            return self.fit(X, **fit_params).transform(X)
        else:
            return self.fit(X, y, **fit_params).transform(X)
```

- `ClassifierMixin` and `TransformerMixin` are abstract concepts — they define what a classifier/transformer should do
- They don't implement `fit()` or `predict()` — these are abstract and must be implemented by child classes
- The `fit_transform()` method provides a default implementation but hides the complexity of when to call fit vs `fit_transform`

Summary Table

Abstraction Type	File Name	Class/Function	What it hides!
ABC	`base.py`	`BaseEstimator`	Parameter handling, serialization, representation
Abstract Mixin	`base.py`	`ClassifierMixin`	Classifier-specific tagging
Abstract Mixin	`base.py`	`TransformerMixin`	fit_transform default implementation
Method Abstraction	`pipeline.py`	`Pipeline.fit_transform`	Sequential transformer fitting
Internal Method	`pipeline.py`	`Pipeline._fit`	Caching, cloning, step iteration
Solver Abstraction	`_logistic.py`	`LogisticRegression.fit`	6 different optimization algorithms
Internal Function	`_logistic.py`	`_logistic_regression_path`	Path-wise optimization for multiple Cs
Abstract Class	`_logistic.py`	`LinearModel`	Abstract fit method with concrete predict
Internal Function	`_base.py`	`_preprocess_data`	Data centering, scaling, validation
Internal Function	`_base.py`	`_rescale_data`	Sample weight application for sparse/dense
Internal Method	`_base.py`	`_set_intercept`	Intercept calculation from centered data

4.4 Encapsulation

Encapsulation is an Object-Oriented Programming concept that bundles data (attributes) and methods that operate on that data within a single unit (class), and restricts direct access to some of an object's internal components. In Python, encapsulation is implemented using:

1. Naming conventions — Single underscore `_attribute` indicates "protected" (internal use)
2. Name mangling — Double underscore `__attribute` creates a private attribute that is harder to access from outside
3. Properties **(**`@property`**)** — Controlled access to attributes with getters/setters
4. Internal methods — Methods starting with _ that are not part of the public API

This prevents external code from directly modifying internal state that could break the object's behaviour.

Example: 01

Internal `_validate_params` Method `base.py (Line 533)`

```
class BaseEstimator:
    def _validate_params(self):
        """Validate types and values of constructor parameters
        This is an internal method (starting with _) that users should not call directly.
        It encapsulates the validation logic.
        """
        validate parameter constraints(
            self._parameter_constraints,
            self.get_params(deep=False),
            caller_name=self.__class__.__name__,
        )
```

- `_validate_params()` is prefixed with a single underscore → internal/encapsulated method
- Users should not call this directly — it's called internally by `_fit_context` decorator
- Encapsulates the complex logic of parameter validation

Example: 02

Protected Attribute `_parameter_constraints` `base.py (Line 533)`

```
class BaseEstimator:
    # _parameter_constraints is a class-level dictionary defining valid parameter types
    # It is encapsulated (underscore prefix) because it's for internal validation only
    _parameter_constraints: dict = {}
    def _validate_params(self):
        validate_parameter_constraints(
            self._parameter_constraints,  # Accessed internally
            ...
        )
```

- `_parameter_constraints` are a protected class attribute (single underscore)
- It encapsulates the rules for valid parameter types and values
- External code should not access or modify it directly — it's for internal validation

Summary Table

Encapsulation Type	File Name	Class/Function	What it Encapsulates
Internal method (_method)	base.py	BaseEstimator._validate_params	Parameter validation logic
Protected attribute (_attribute)	base.py	BaseEstimator._parameter_constraints	Parameter constraint rules
Internal classmethod	base.py	BaseEstimator._get_param_names	Parameter name discovery via reflection
Internal method	pipeline.py	Pipeline._fit	Sequential transformer fitting
Internal generator	pipeline.py	Pipeline._iter	Step iteration with filtering
Internal helper	pipeline.py	Pipeline._log_message	Log message formatting

Internal routing	pipeline.py	Pipeline._check_method_params	Parameter routing logic
Internal function	_logistic.py	_check_solver	Solver compatibility validation
Internal function	_logistic.py	_logistic_regression_path	Path-wise optimization algorithm
Protected attribute	_logistic.py	LogisticRegression._parameter_constraints	LR parameter validation rules
Internal method (inherited)	_base.py	LinearClassifierMixin._predict_proba_lr	Binary probability calculation
Internal function	_base.py	_preprocess_data	Data centering & validation
Internal function	_base.py	_rescale_data	Sample weight rescaling
Internal method	_base.py	LinearModel._set_intercept	Intercept calculation

5. Design Analysis

The design of **scikit-learn** is centered on a uniform, consistent API that prioritizes ease of use and interoperability with the broader Python scientific stack (NumPy, SciPy, and Matplotlib). Its architecture is built around three core interfaces: Estimators, Predictors, and Transformers.

Core Design Principles

The library subscribes to a few principles that enable it to be open and effective:

Principle 1 — Consistency

All objects have the same interface; that is, they have the same methods (such as fit or predict). Runs multiple algorithms.

```
# Same interface for every model
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
model1 = LogisticRegression()
model2 = DecisionTreeClassifier()
# Exact same interface — no need to learn new methods
model1.fit(X_train, y_train)
model2.fit(X_train, y_train)
model1.predict(X_test)
model2.predict(X_test)
```

Principle 2 — Inspection

There are some parameters determined by the model and others learned (identifiers with a trailing), The public attributes (underscore, e.g., `coef_`) contain these.

```
model = LogisticRegression(max_iter=200)
model.fit(X, y)
# User-set hyperparameters — no underscore
print(model.max_iter)       # 200
# Learned parameters — trailing underscore
print(model.coef_)          # learned coefficients
print(model.intercept_)     # learned intercept
print(model.n_iter_)        # number of iterations taken
```

Principle 3 — Non-proliferation of Classes

In scikit-learn, the datasets are intentionally not programmed as specific classes. Rather, it provides use of standard Arrays (NumPy) and sparse matrices (Scipy) Only the learning algorithms are shown by custom classes. This ensures that the library does not require too much memory and can fit into the whole python data. science ecosystem.

```
import numpy as np
# Standard NumPy arrays — no custom dataset class needed
X = np.array([[1, 2], [3, 4], [5, 6]])
y = np.array([0, 1, 0])
model = LogisticRegression()
```

```
model.fit(X, y)   # Works directly with NumPy arrays
```

Principle 4 — Sensible Defaults

All the scikit-learn algorithms use a set of tuned default parameters for optimal performance on Most typical scenarios. Help the beginner get started right away without having to understand every parameter:

```
# Works perfectly with default parameters
model = LogisticRegression()
model.fit(X, y)
predictions = model.predict(X_test)
```

Primary Interfaces

Given these design principles, all the objects in Scikit-learn can be primarily divided into three interface groups. types:

Interface	Purpose	Primary Methods
Estimators	object that learns from data — classifiers, regressors	`fit(X, y)`
Transformers	Estimators that can transform a dataset — scaling, normalization	`transform(X)`, `fit_transform(X)`
Predictors	Estimators that make predictions on new data	`predict(X)`,`predict_proba(X)`

Workflow Architecture in a typical Scikit-learn:

The design enables to apply the standard machine learning workflow by using special modules:

- Preprocessing: Feature scaling (such as StandardScalar), normalization. Filling in missing data (e.g., SimpleImputer).
- Model Selection: Utilities for comparing and validating models, such as GridSearchCV for hyperparameter tuning and cross-validation.
- Evaluation: A comprehensive Scoring API that includes metrics for classification (accuracy, F1-score), regression (R^2), and clustering

Conclusion of Design Analysis

End of Design Analysis (successful) It is easy to see how the developers of Scikit-learn have thought through the design decisions they tackled.A true deep understanding of Object can be seen in the design decisions that the Scikit-learn developers took. Oriented Design principles. The Mixin pattern can be used with the four fundamental design principles of Based on Consistency, Inspection, Non-proliferation, and Sensible Defaults, Scikit-learn is the most Most popular machine learning Library in Python. While the Mixin pattern introduces complexity. It is true that the benefits of flexibility, reusability and maintainability outweighed with multiple inheritance, but there is a price to pay as well. outweigh the disadvantages. It's an example in the textbook that professional developers use! Real-life software using OOP principles.

6. Comparison with Alternative Library

Both Scikit-learn and PyTorch are popular machine learning libraries in Python. Their Object-Oriented designs are quite different though. Scikit-learn The products differ in their emphasis: focus on traditional machine learning algorithms, PyTorch is focused on deep learning. using neural networks. This section compares the OOP design of both libraries.

Base Class Comparison

Scikit-learn:

```
# Every model inherits from BaseEstimator
```

```
from sklearn.base import BaseEstimator, ClassifierMixin
class LogisticRegression(ClassifierMixin, BaseEstimator):
    def fit(self, X, y): ...
    def predict(self, X): ...
```

PyTorch:

```
# Every model inherits from nn.Module
import torch.nn as nn
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
    def forward(self, x): ...
```

Key Difference: Scikit-learn uses `BaseEstimator` as the root class for all models, while PyTorch uses `nn.Module`. Both follow inheritance — but the interface is different. Scikit-learn uses `fit()` and `predict()`, while PyTorch uses `forward()`.

Polymorphism Comparison

Scikit-learn — Polymorphism through Mixin classes:

```
# Same method name — different behavior
class ClassifierMixin:
    def score(self, X, y):
        return accuracy_score(y, self.predict(X))  # Accuracy
class RegressorMixin:
    def score(self, X, y):
        return r2_score(y, self.predict(X))  # R2 Score
```

PyTorch — Polymorphism through forward() override:

```
# Same method name — different behavior
class CNN(nn.Module):
    def forward(self, x):
        # Convolutional operations
        return x
class RNN(nn.Module):
    def forward(self, x):
        # Recurrent operations
        return x
```

Both libraries achieve polymorphism — but in different ways. Scikit-learn uses the Mixin pattern where different Mixin classes provide different implementations of `score()`. PyTorch uses method overriding where each model overrides the `forward()` method differently.

Composition Comparison

Scikit-learn — Pipeline:

```
from sklearn.pipeline import Pipeline
# Compose multiple steps into one object
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression())
])
pipeline.fit(X_train, y_train)
pipeline.predict(X_test)
```

PyTorch — nn.Sequential:

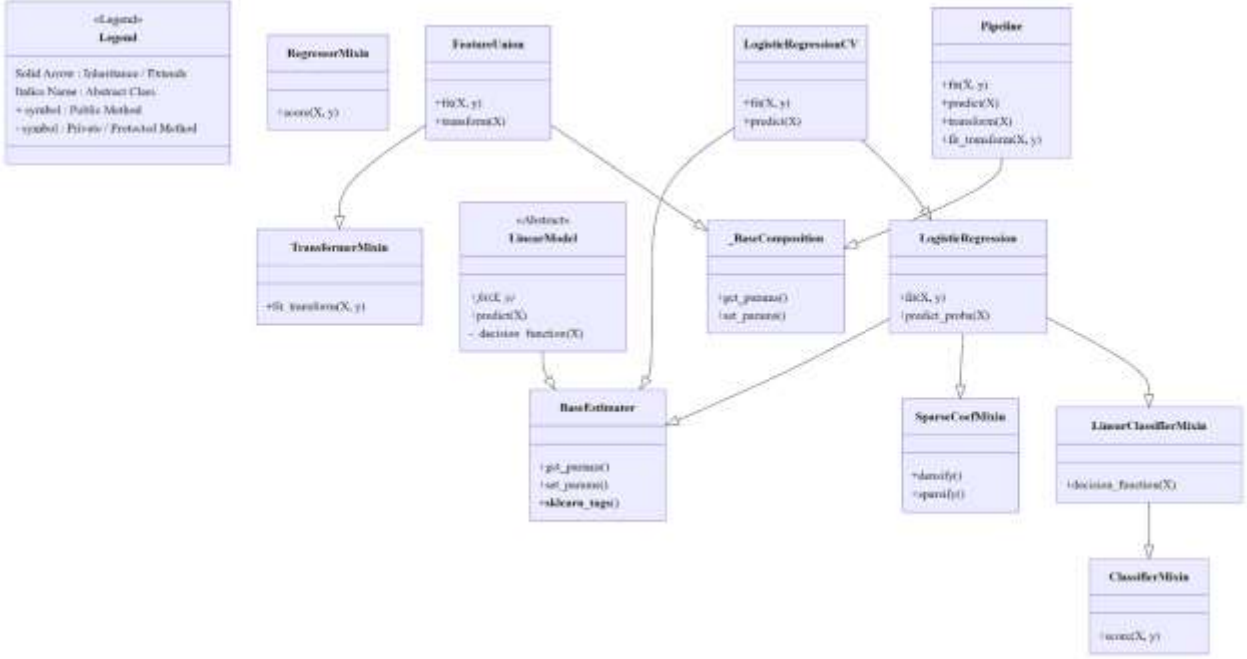
```
import torch.nn as nn
# Compose multiple layers into one object
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)
```

Both use Composition to chain multiple components together. Scikit-learn's `Pipeline` chains preprocessing steps and estimators, while PyTorch's `nn.Sequential` chains neural network layers. The concept is the same — the implementation is different. Scikit-learn's OOP design is more structured and consistent — making it ideal for traditional machine learning where a uniform interface is important. PyTorch's design is more flexible — making it ideal for research and deep

learning where custom architectures are needed. Both are excellent examples of real-world OOP design in Python.

7. UML Diagram

The following UML Class Diagram illustrates the key inheritance hierarchy, mixins, and relationships among the major classes analyzed from the Scikit-learn source code (`base.py, pipeline.py, _base.py, and _logistic.py`).



UML Class Diagram represents Inheritance and Mixins as well as Composition in Scikit-learn.
Key Observations from the UML Diagram:

- BaseEstimator is a base class for almost all estimators in scikit-learn, and serves as a root class. It provides common functionality such as parameter handling, validation, and configuration management..Offers common functionality like get_params(), set_params(), and parameter overloads without changing the data semantics for each bundle. validation.
- Mixin Pattern is extensively used (ClassifierMixin, RegressorMixin, TransformerMixin, SparseCoefMixin, LinearClassifierMixin). This allows reusable behaviour so that it can be added to several classes without the need to have too many inheritance levels.
- Multiple Inheritance has been perfectly illustrated in classes such as LogisticRegression and FeatureUnion.
- Composite Design Pattern: Pipeline/FeatureUnion is used to demonstrate. Complex workflows to be viewed as single estimators.
- Abstract Class (LinearModel with ABCMeta) – impose a contract on all linear models by Defining common functions such as predict(), _decision_function(), etc.
- No difference in public interface (fit(), predict(), transform(), score()) between all classes is the foundation of scikit-learn's user-friendly design.

The pattern is clearly understood due to this diagram and the concept of code reusability, code consistency is understood. Extensibility using good OOP practices.

8. Custom Python Extension

In this section, we extend Scikit-learn's existing classes by creating three custom classes that demonstrate all major OOP concepts. Each class inherits from at least one Scikit-learn base class and adds meaningful new functionality.

Overview of Custom Classes:

Class	Inherits From	New Methods Added
SmartClassifier	`ClassifierMixin` + `BaseEstimator`	`describe()`, `get_model_info()`, `compare_strategies()`
SmartTransformer	`TransformerMixin` + `BaseEstimator`	`describe()`, `get_transformer_info()`
SmartPipeline	None — Composition	`describe()`, `get_pipeline_info()`, `score()`

Class 1: SmartClassifier

`SmartClassifier` inherits from two Scikit-learn base classes — `ClassifierMixin` and `BaseEstimator`. It overrides the `fit()` and `predict()` methods and adds three completely new methods that do not exist in any parent class.

```
from sklearn.base import BaseEstimator, ClassifierMixin
class SmartClassifier(ClassifierMixin, BaseEstimator):
    def __init__(self, strategy='majority', threshold=0.5):
        self.strategy  = strategy    # Encapsulation
        self.threshold = threshold
    def fit(self, X, y):             # Polymorphism — overridden
        self.classes_         = np.unique(y)
        self.majority_class_ = int(np.argmax(np.bincount(y.astype(int))))
        self.is_fitted_       = True
        return self
    def predict(self, X):            # Polymorphism — overridden
        return np.full(len(X), self.majority_class_)
    def describe(self):              # NEW method — not in parent
        print(f"Strategy : {self.strategy}")
        print(f"Classes  : {self.classes_}")
```

Class 2 — SmartTransformer

`SmartTransformer` inherits from `TransformerMixin` and `BaseEstimator`. It overrides `fit()` and `transform()` to perform data scaling, and adds new methods for description and information retrieval.

```
from sklearn.base import BaseEstimator, TransformerMixin
class SmartTransformer(TransformerMixin, BaseEstimator):
    def __init__(self, method='standard'):
        self.method = method
    def fit(self, X, y=None):        # Polymorphism — overridden
        self.mean_  = np.mean(X, axis=0)   # Encapsulation
        self.std_   = np.std(X, axis=0)
        self.is_fitted_ = True
        return self
    def transform(self, X):          # Polymorphism — overridden
        return (X - self.mean_) / (self.std_ + 1e-8)
    def describe(self):              # NEW method — not in parent
        print(f"Method : {self.method}")
        print(f"Mean   : {self.mean_}")
```

Class 3 — SmartPipeline

`SmartPipeline` composes both `SmartTransformer` and `SmartClassifier` into one complete workflow. This demonstrates the Composition design pattern — where one class contains objects of other classes.

```
class SmartPipeline:

    def __init__(self, transformer, classifier):
        # Composition — objects stored inside
        self.transformer = transformer
        self.classifier  = classifier
```

```
    def fit(self, X, y):              # Abstraction — steps hidden
        X_transformed = self.transformer.fit_transform(X, y)
        self.classifier.fit(X_transformed, y)
        return self
    def predict(self, X):             # Abstraction — steps hidden
        X_transformed = self.transformer.transform(X)
        return self.classifier.predict(X_transformed)
    def score(self, X, y):            # NEW method
        return np.mean(self.predict(X) == y)
```

The complete source code is available in the GitHub repository at
https://github.com/muhammadbilal744/Scikit-Learn-OOP-Analysis/blob/main/Code/custom_extention.py

9. Conclusion

This project provided an in-depth Object-Oriented Design analysis of the Scikit-learn library — one of the most widely used machine learning libraries in Python. By studying the source code of four core files — `base.py`, `pipeline.py`, `_logistic.py`, and `_base.py` — we were able to identify and understand how professional developers apply OOP principles in real-world software.

What We Learned

Through this analysis, we discovered that Scikit-learn is built on a carefully designed class hierarchy that demonstrates all four major OOP principles:

- Inheritance was extensively used across all analyzed source files. Classes like `LogisticRegression` inherit from multiple parent classes simultaneously — `LinearClassifierMixin`, `SparseCoefMixin`, and `BaseEstimator` — demonstrating Multiple Inheritance in a real-world library.
- Polymorphism was clearly visible in the `score()` method — which behaves differently in `ClassifierMixin` and `RegressorMixin` — and in the `fit()` method which is overridden differently in every model class.
- Abstraction was demonstrated through methods like `fit_transform()` in `TransformerMixin`, which hides the internal steps from the user, and `Pipeline.fit()`, which runs multiple steps internally without exposing the details.
- Encapsulation was evident through private methods like `_validate_steps()` and `_fit()` in `pipeline.py`, and through the naming convention of learned attributes with trailing underscores such as `coef_` and `intercept_`.

Design Analysis Findings

The most significant design decision in Scikit-learn is the use of the Mixin Pattern. Instead of creating one large God Class, the developers split functionality into small focused Mixin classes — `ClassifierMixin`, `RegressorMixin`, and `TransformerMixin`. This is a decision that follows the following: Single Responsibility Principle and makes the library flexible, reusable and extendable. This is a best practice textbook of how SOLID principles are used in professional software development.

Custom Extension

We implemented three new custom classes based on the classes of Scikit-learn in this project:

- `SmartClassifier` — inherits from `ClassifierMixin` and `BaseEstimator`
- `SmartTransformer` — inherits from `TransformerMixin` and `BaseEstimator`
- `SmartPipeline` — composes both classes using the Composition pattern

These classes provide an illustration on how easily the capability of Scikit-learn can be extended by adding new functionality to a class. Simple addition - by simply inheriting from the overall base classes without alteration of any original code.

Final Thoughts

This project served as a bridge between theoretical OOP concepts and their real-world implementation, covered in the texts and how it is actually used professional Python libraries. Scikit-learn is not only a machine learning library, but also an excellent example of professional object-oriented software design. Its architecture demonstrates how abstraction, inheritance, polymorphism, and encapsulation can be applied to create scalable and maintainable software systems. Explain the source code and gave us more understanding about its source code. Understanding Abstraction, Inheritance, Polymorphism and Encapsulation — and how they work. Working collaboratively to develop consistent, flexible and maintainable software.

10.Future Work

This project focused on the Object-Oriented Design analysis of selected Scikit-learn source code files, including base.py, pipeline.py, _logistic.py, and _base.py. In future work, deeper analysis can be performed on additional Scikit-learn modules such as ensemble learning, preprocessing, clustering, and neural network architectures. More advanced UML diagrams can also be developed to represent the complete architecture of the library.

Furthermore, the custom implementation created in this project can be extended by adding real machine learning functionality, additional transformers, and model evaluation techniques. Future projects may also compare Scikit-learn's architecture with other machine learning libraries such as TensorFlow and PyTorch to better understand different software design approaches used in professional AI frameworks.

11. References

[1] Scikit-learn Official Documentation. *Scikit-learn: Machine Learning in Python.*
Available at: https://scikit-learn.org/stable/
[2] Scikit-learn GitHub Repository. *Source code of Scikit-learn library.*
Available at: https://github.com/scikit-learn/scikit-learn
[3] Scikit-learn — base.py Source Code. *BaseEstimator, ClassifierMixin, RegressorMixin, TransformerMixin.*
Available at: https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/base.py
[4] Scikit-learn — pipeline.py Source Code. *Pipeline, FeatureUnion.*
Available at: https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/pipeline.py
[5] Scikit-learn — _logistic.py Source Code. *LogisticRegression, LogisticRegressionCV.*
Available at: https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/linear_model/_logistic.py
[6] Scikit-learn — _base.py Source Code. *LinearModel, LinearRegression,LinearClassifierMixin.*
Available at: https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/linear_model/_base.py
[7] Python Official Documentation. *inspect module — Source code inspection.*
Available at: https://docs.python.org/3/library/inspect.html
[8] PyTorch Official Documentation. *torch.nn.Module — Base class for neural networks.*
Available at: https://pytorch.org/docs/stable/generated/torch.nn.Module.html
[9] Wikipedia. *Scikit-learn — Overview and History.*
Available at: https://en.wikipedia.org/wiki/Scikit-learn
[10] Brennan, M. A. *Scikit-learn Design Principles.*
Available at: https://medium.com/@markabrennan/scikit-learn-design-principles